What Makes a Virtual ISA Agent-Programmable?

A Controlled Study of Agent-Facing Documentation for StableHLO

Luxin Zhang
Independent Researcher
luxin.zhang92@outlook.com

***Abstract*—AI agents increasingly write code at the virtual instruction-set architecture (ISA) layer, while benchmarks score the agent, not the *interface* the ISA exposes. Isolating that interface is hard because documentation variants must carry the same facts, examples and length. We fix one intermediate representation (IR), StableHLO, with a pinned verifier and executor, the tasks and a harness-guided generate-and-repair loop, and vary only how documentation reaches the agent: none, syntax only, prose, a length-matched JSON contract with the same facts and examples, or retrieval through a tool (free, forced, or with cross-cutting rules upfront). With a protocol frozen beforehand, we evaluated 267 held-out tasks (114 synthetic, 153 lowered from KernelBench PyTorch modules) with independent oracles and three GPT-4.1 snapshots (5,005 trials; the largest model on 91 tasks). Even syntax-only documentation added 16 to 33 percentage points (pp) of first-attempt correctness over none. With matched content, the tested JSON contract scored below prose for every model (-6.0, -7.9, -12.1 pp; significant for the two larger), reversing our earlier, unmatched comparison; an ablation splits the gap between the JSON wrapper and formal predicates. On-demand access trailed upfront delivery by 4–32 pp: the strongest model never called the tool unless forced, partial upfront documentation suppressed lookup, and agents often missed the cross-cutting encoding rule. Over 99% of observed failures were rejected statically, even on real-world tasks, and a stronger oracle changed 1 of 1,069 accepted first attempts: well-formedness was the binding bottleneck. All artifacts will be released.**

***Index Terms*—virtual ISA, StableHLO, MLIR, LLM agents, agent-facing interfaces, documentation, code generation, benchmarking**

I. Introduction

A virtual ISA decouples programs from rapidly changing hardware: front ends target the virtual layer and back ends absorb the churn [1], [2]. Increasingly, the producer at that layer is an AI agent that writes, repairs and lowers code, which makes the presentation of the ISA to a non-human programmer a design dimension in its own right. Existing benchmarks score models on GPU kernels [3]–[5], LLVM IR optimization [6] or unfamiliar accelerator ISAs [7]; they answer "how good is the agent?" for a given interface. We ask the complementary question: *for a fixed agent and a fixed IR, which properties of the agent-facing interface change whether the agent writes correct programs?*

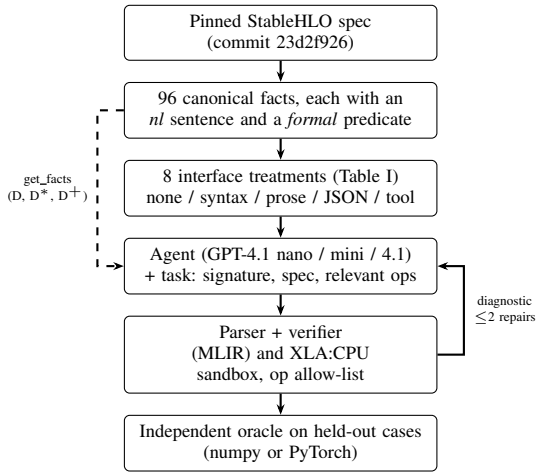
Isolating the interface is harder than it looks, for three reasons. First, documentation variants must be information-equivalent: if one rendering carries a concrete example and another only a schema, or one is longer, the comparison measures that difference rather than the representation. Our own first study fell into this trap and reached a conclusion that later reversed (§IV). Second, correctness must be judged by oracles that are independent of the IR, including for real-world code lowered from ML frameworks. Third, making documentation available is not the same as having it used: an agent with a retrieval tool may never call it.

We address these challenges with a fact-level design on StableHLO [8], a versioned, specification-backed dialect of MLIR (Multi-Level IR) [2] that serves as a portability layer between ML frameworks and compilers (Fig. 1). A canonical inventory anchored to the pinned specification is rendered as length-matched prose, as a formal contract with the same examples, or behind a retrieval tool whose uptake we measure and, in one condition, force. Correctness is judged by numpy or PyTorch oracles on held-out inputs. In our data the answer is dominated by *well-formedness*: agents rarely wrote programs that verified but computed the wrong values. Our claims therefore concern LLM program generation with harness-guided repair against one pinned StableHLO toolchain, and the learnability of an ISA's surface and static rules; they are not claims about semantic reasoning, fully autonomous agents, or ISA design beyond its documentation.

Contributions. (i) A controlled methodology and harness that vary agent-facing ISA documentation while holding IR, toolchain, tasks and agent loop fixed, with an automated equivalence audit (§III). (ii) A 328-task StableHLO suite: synthetic tasks plus 176 KernelBench modules lowered and validated end-to-end against PyTorch oracles (§III-C). (iii) Two studies: an exploratory one that exposed confounds in our own treatments, and a confirmatory one with a frozen protocol (5,005 trials) that removes them (§IV). (iv) Design implications for agent-facing virtual ISAs, including results that reverse the ones we obtained before matching the treatments (§V).

II. Related Work

Kernel and IR benchmarks fix the interface and compare agents. KernelBench [3] evaluated LLM generation of efficient GPU kernels from PyTorch references, TritonBench [4] evaluated the generation of Triton operators from real-world and PyTorch-aligned tasks, and TritonGym [5] added agentic tool use to separate model ability from workflow design. CIRBench [6] evaluated LLMs on analyzing, repairing and optimizing LLVM IR, and KernelCraft [7] benchmarked agents writing kernels for emerging accelerator ISAs. Schema-derived constrained decoding addressed natural-language-to-



Task (`bias_add_col`): $out[i,j] = x[i,j] + b[i]$, `@main(%x: tensor<4x2xf32>, %b: tensor<4xf32>)`.

Fact MOD.SYN.3, prose (B): "One-dimensional integer-list attributes (dimensions, permutation, ...) are written like `array<i64: 0, 1>`, never in the older `dense<[0, 1]> : tensor<2xi64>` form; ..."

Same fact, structured (C): `{"id":"MOD.SYN.3","rule":"int_list_attr := 'array<i64: d0, d1, ...>' (NOT 'dense<[...]> : tensor<Nxi64>'); ...; example: 'array<i64: 0, 1>'"}`

Agent (mini, D^+: this rule shown upfront), first attempt:
`%0 = "stablehlo.broadcast_in_dim"(%b) {broadcast_dimensions = dense<0> : tensor<1xi64>} ...`

Verifier: `'stablehlo.broadcast_in_dim' op attribute 'broadcast_dimensions' failed to satisfy constraint: i64 dense array attribute`

Repair (accepted by verifier and oracle):
`%0 = "stablehlo.broadcast_in_dim"(%b) {broadcast_dimensions = array<i64: 0>} ...`

Fig. 1. Study design (left) and a running example from the Study 2 logs (right). The same canonical fact is rendered as prose or as a formal JSON rule; the agent's stale prior (the legacy `dense` encoding) overrides the rule even when it is shown, and the verifier's diagnostic, not the oracle, drives the repair, which is the pattern behind F3 and F4.

MLIR generation, including StableHLO, from the decoder side [9], which is complementary to changing the interface.

Interface and documentation studies vary what the model sees. SWE-agent [10] showed that the design of the *agent-computer interface* changes agent success; its interface is a set of tools and actions, whereas ours is the ISA's documentation. DocPrompting [11] showed that retrieved documentation helps code generation for unseen APIs. For PDDL, a low-resource planning language, documentation alone did not help, but pipeline-driven retrieval combined with modular generation and error refinement did [12]; in our study the *agent* drives retrieval, and the outcome hinges on uptake and on which facts it requests. Code models carry stale knowledge of evolving APIs [13], the mechanism behind the legacy attribute encodings that StableHLO's verifier rejects in our logs.

Prompt-format studies showed that semantically equivalent formatting changes can swing LLM accuracy widely, with weak correlation across models [14], and that plain-text, Markdown, JSON and YAML templates change GPT-model results, with larger models more robust [15]. Our F2 is consistent with format sensitivity, but in our data the format penalty did not shrink with model size. We bring this interface view to the virtual ISA and manipulate the representation and access mode of its documentation at the level of fact-equivalent treatments.

III. Study Design

This section describes the fixed components, the documentation treatments, the tasks and oracles, and the statistical plan (Fig. 1, left). We ran two studies. **Study 1** (exploratory; 114 synthetic tasks, 2,052 trials) surfaced two confounds in our own treatments and an uptake failure in on-demand documentation. **Study 2**, which the rest of this paper reports, was pre-registered internally after Study 1: its protocol, task manifest and code were frozen and hashed in version control before any evaluation trial. It fixes those confounds, adds real-world tasks derived from KernelBench, and turns the Study 1 post-hoc conditions into planned ones. The protocol, file hashes and all trial logs will be released.

A. Fixed IR, toolchain and agent loop

All conditions share one toolchain and one agent loop. Programs are StableHLO modules with a single `@main`. We parse and verify them with the MLIR/StableHLO bindings in jaxlib 0.11.2 (StableHLO 1.20.0) and execute them on the XLA compiler's CPU backend (XLA:CPU) in a sandboxed subprocess with a 60 s timeout. An operation allow-list admits StableHLO and three element-wise operations from CHLO, the companion "client" dialect, and excludes `custom_call`. The task fixes the signature exactly. The agent receives a system prompt, the documentation (if any) and the task, and replies with a program. Failures are returned verbatim (the verifier diagnostic, or the first mismatching oracle element), with up to two repairs. Documentation stays in context, and in the interactive conditions the retrieval tool remains available for up to 6 calls. Each trial is a fresh conversation at temperature 0, one per task and condition.

B. Canonical facts and interface treatments

The treatments differ only in how one fact inventory reaches the agent. The inventory has 96 facts over 23 operation groups (25 syntax, 16 type, 32 shape, 23 semantics). Each fact is anchored to a clause of the pinned specification [8] and authored twice: as a natural-language sentence and as a formal predicate in specification notation (Fig. 1, right). Table I lists the eight treatments.

Two changes remove the Study 1 confounds. First, every syntax fact now carries the same general schema and the same concrete instance in both renderings (e.g. both `array<i64: ...>` and `array<i64: 1>`); in Study 1, prose carried only the instance and the formal rendering only the schema. Second, the structured contract is compact JSON, so its mean length is $0.98\times$ that of prose over all 328 tasks (Study 1: $1.31\times$), measured with the o200k tokenizer. An automated audit checks

TABLE I
INTERFACE TREATMENTS (STUDY 2). TOK.: MEAN DOCUMENTATION TOKENS (O200K TOKENIZER) OVER 328 TASKS. †POST-HOC ABLATION (SEC. IV).

	Condition	Facts upfront	Rendering	Tok.
–	NONE	none	–	0
A	MINIMAL	syntax	prose	951
N	SYNTAX-ONLY STR.	syntax	JSON	850
B	PROSE	all	prose	1455
C	STRUCTURED	all	JSON + IDs	1469
C′	JSON-PROSE†	all	JSON, prose	1807
D	INTERACTIVE	none (on demand)	JSON, tool	0
D*	INT. FORCED	as D, 1st call forced	JSON, tool	0
D+	INT. CORE	module; rest on demand	JSON, tool	362

every task: identical fact-ID sequences, each rendering present verbatim, and no IDs leaking into prose. All 29 code examples verify with the pinned toolchain. A task's fact set is the cross-cutting `module` facts (program structure, types, the integer-list attribute encoding, no implicit broadcasting) plus the facts of the operation groups the task lists as relevant. Every condition receives the identical task text, signature and list of relevant groups.

C. Tasks and independent oracles

The evaluation population combines controlled synthetic tasks with real-world tasks lowered from PyTorch.

Synthetic (114 tasks). We wrote 38 parametric templates over broadcasting, reductions, contractions, layout, slicing and padding, masking and indexing, conversion, and composites (e.g. softmax). Each template is instantiated three times with distinct dimension sizes, and one additional instance per template forms a development split. Each task is specified mathematically. Its oracle is a numpy reference evaluated on three held-out cases that include semantic traps: all-negative rows for max-reductions, exact zeros for strict comparisons, and out-of-range `dynamic_slice` starts.

KernelBench-derived (153 tasks). We converted all 200 level-1 and level-2 tasks of KernelBench [3] (MIT license). They cover single operators, normalizations, pooling, convolutions, losses, and fused operator chains. The pipeline (i) shrinks integer size literals deterministically, preserving divisibility where required; (ii) exposes all module parameters as explicit `@main` arguments, keeping the training-mode semantics under which KernelBench evaluates; (iii) uses PyTorch eager outputs on three cases, each with freshly initialized parameters and inputs, as the oracle ($\mathrm{rtol}=10^{-3}$); and (iv) produces a reference StableHLO program via `torch.export` and `torchax`, a PyTorch-to-JAX bridge. The reference is never shown to agents; it only demonstrates that the task is expressible and that the oracle and toolchain agree. The agent sees the PyTorch module source, the argument mapping and the signature. We excluded 24 tasks, each with a logged reason: 18 were too large even after shrinking, 2 had shape couplings broken by shrinking, 2 use dropout, 1 takes a non-tensor input, and 1 failed validation because its reference disagreed with PyTorch. Of the remaining 176, a 10% hash split (23 tasks) forms the development set, leaving 153 evaluation tasks and a total evaluation population of 267 tasks.

Validation (gate G1). All 328 reference programs (152 synthetic, 176 KernelBench) parse, verify, execute and match their oracles. Mutating the reference programs (operator swaps, constant and attribute perturbations) showed that the oracle rejected all 132 non-equivalent executing mutants on synthetic tasks, but only 322 of 432 executing mutants on KernelBench tasks, whose original inputs are non-negative. We therefore built a stronger oracle: five new cases per task, judged by PyTorch, with signed, wide-range, zero-heavy, outlier and all-negative inputs (non-negative for one task whose operation requires it); two tasks whose reference fails these cases were set aside. It killed 9 further mutants; the remaining 95 also survive it and are likely equivalent. Re-grading all 1,069 accepted first attempts and 1,282 accepted final attempts changed 1 and 3 labels, and none of the 24 pre-registered contrasts by more than 0.5 pp. We report the pre-registered grading.

D. Models, budget and statistics

We used three dated snapshots of one model family, `gpt-4.1-nano`, `gpt-4.1-mini` and `gpt-4.1` (all `2025-04-14`), called *nano*, *mini* and *4.1* below. A local Qwen2.5-7B solved 0 of 12 development trials and was not used. The total API budget was capped at $35 and enforced by the runner. Trials were ordered task-blocked (tasks shuffled, conditions shuffled within each task), so an outcome-blind budget stop leaves complete, paired tasks. The primary endpoint is first-attempt semantic correctness, and the primary contrast is C−B. Eight paired contrasts were pre-specified (Table III). For each model we report the mean paired difference in percentage points (pp) with a 95% cluster-bootstrap CI (10k resamples; clusters are templates for synthetic tasks and individual tasks for KernelBench), together with exact McNemar tests, Holm-corrected within each model. Per-suite results are secondary and Holm-corrected within each suite. Because 4.1 covers a task-blocked random subset, we checked that subset on nano and mini: it has the same suite mix (57% KernelBench) and is slightly harder (prose 36% vs. 41% for mini). The failure taxonomy (assembly syntax, attribute encoding, literal, shape/type, unknown op, wrong values) was fixed before the runs.

IV. RESULTS

This section reports the four findings for the pre-specified endpoints. All pre-registered trials completed for nano and mini (267 tasks each, 2,136 trials per model). The 4.1 run reached its share of the budget after 733 trials; under the frozen task-blocked rule it is analyzed on its 91 complete tasks (39 synthetic, 52 KernelBench). The evaluation runs cost $34.52, plus $0.04 for a development pilot. Fig. 2 shows per-suite correctness, Table II the overall rates and Table III the pre-specified contrasts.

F1: Documentation was the dominant lever. Syntax facts alone, the smallest documentation condition (A−None), added $+15.7$, $+32.6$ and $+27.5$ pp (all Holm $p < 0.001$). Without

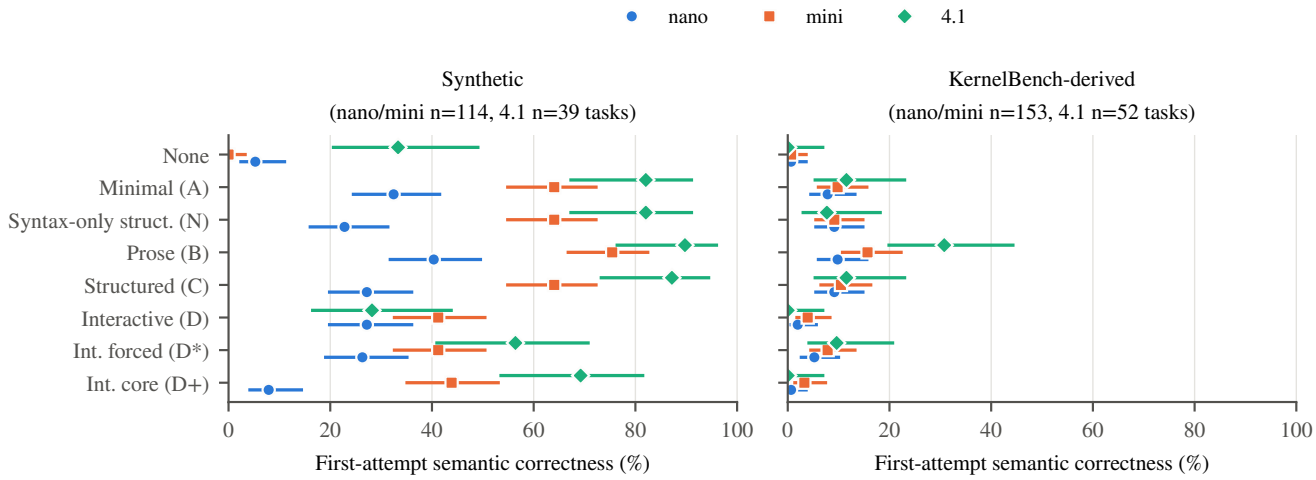


Fig. 2. Documentation dominates, prose scores at or above the structured contract, and on-demand conditions trail upfront delivery. First-attempt semantic correctness by interface condition and task suite (95% Wilson intervals). Nano and mini cover all tasks; 4.1 covers the 39 synthetic and 52 KernelBench tasks completed within budget.

TABLE II
SEMANTIC CORRECTNESS (%) OVER ALL EVALUATION TASKS: FIRST ATTEMPT (1ST) AND AFTER ≤2 REPAIRS (REP.). n = 267 TASKS FOR NANO AND MINI, 91 FOR 4.1.

	nano		mini		4.1	
Condition	1st	rep.	1st	rep.	1st	rep.
None	2.6	3.4	0.4	0.7	14.3	19.8
Minimal (A)	18.4	19.9	33.0	40.4	41.8	48.4
Syntax-only struct. (N)	15.0	17.2	32.6	40.8	39.6	47.3
Prose (B)	22.8	24.7	41.2	48.7	56.0	60.4
Structured (C)	16.9	23.2	33.3	39.7	44.0	53.8
Interactive (D)	12.7	13.5	19.9	21.7	12.1	16.5
Int. forced (D*)	14.2	15.7	22.1	23.6	29.7	31.9
Int. core (D+)	3.7	4.5	20.6	34.8	29.7	37.4

documentation, mini solved 1 of 267 tasks. Semantic and shape facts on top of syntax (B−A) added +4.5 (n.s.), +8.2 and +14.3 pp, growing with capability.

F2: With matched content, the tested structured contract scored below prose. The primary contrast C−B was negative for every model: −6.0 pp [−12.5, 0.0] for nano (Holm $p = 0.083$, n.s.), −7.9 pp [−12.6, −3.7] for mini ($p < 0.001$) and −12.1 pp [−19.1, −5.7] for 4.1 ($p = 0.004$, 11 vs. 0 discordant tasks).

Where the penalty fell depended on the model. For 4.1 it was concentrated on KernelBench (30.8% with prose vs. 11.5% with the contract, −19.2 pp, $p = 0.016$ with Holm correction within the suite), while on synthetic tasks the two were within 3 pp (89.7% vs. 87.2%). For nano and mini it was instead larger on synthetic tasks (−13.2 and −11.4 pp) than on KernelBench (−0.7 and −5.2 pp), where their accuracy was low overall. The format of syntax facts alone made no difference (N−A: −3.4, −0.4, −2.2 pp, all n.s.). C differs from B in its JSON wrapper, fact IDs and grouping by category as well as in its formal predicates; the ablation below

TABLE III
PRE-SPECIFIED PAIRED CONTRASTS, FIRST ATTEMPT, ALL TASKS. Δ: FIRST MINUS SECOND CONDITION IN EACH PAIR, IN PP; 95% CLUSTER-BOOTSTRAP CI; DISC. = TASKS SOLVED ONLY BY THE FIRST / ONLY BY THE SECOND CONDITION; HOLM WITHIN MODEL.

Model	Contrast	Δ (pp)	95% CI	disc.	p_{Holm}
nano	Structured – Prose	-6.0	[-12.5, +0.0]	12/28	0.083
	Interactive – Structured	-4.1	[-9.0, +0.7]	11/22	0.240
	Int. forced – Interactive	+1.5	[-0.8, +4.1]	9/5	0.424
	Int. core – Interactive	-9.0	[-14.7, -3.8]	3/27	<0.001
	Int. core – Structured	-13.1	[-19.8, -6.6]	5/40	<0.001
	Prose – Minimal	+4.5	[+0.0, +9.1]	18/6	0.091
	Minimal – None	+15.7	[+10.1, +21.8]	43/1	<0.001
	Syntax-only struct. – Minimal	-3.4	[-8.6, +1.5]	11/20	0.299
mini	Structured – Prose	-7.9	[-12.6, -3.7]	4/25	<0.001
	Interactive – Structured	-13.5	[-19.3, -8.0]	4/40	<0.001
	Int. forced – Interactive	+2.2	[-1.1, +5.5]	14/8	0.859
	Int. core – Interactive	+0.7	[-5.8, +7.0]	22/20	1.000
	Int. core – Structured	-12.7	[-18.6, -7.2]	4/38	<0.001
	Prose – Minimal	+8.2	[+3.7, +13.3]	26/4	<0.001
	Minimal – None	+32.6	[+24.3, +40.5]	87/0	<0.001
	Syntax-only struct. – Minimal	-0.4	[-4.2, +3.2]	8/9	1.000
4.1	Structured – Prose	-12.1	[-19.1, -5.7]	0/11	0.004
	Interactive – Structured	-31.9	[-44.2, -19.0]	2/31	<0.001
	Int. forced – Interactive	+17.6	[+4.5, +30.6]	21/5	0.007
	Int. core – Interactive	+17.6	[+7.2, +28.7]	17/1	<0.001
	Int. core – Structured	-14.3	[-24.4, -5.4]	2/15	0.007
	Prose – Minimal	+14.3	[+7.5, +21.8]	13/0	0.001
	Minimal – None	+27.5	[+15.2, +39.6]	27/2	<0.001
	Syntax-only struct. – Minimal	-2.2	[-6.8, +2.1]	1/3	0.625

separates these. For 4.1 the extra structured failures were mostly shape/type verifier errors (32 vs. 13 first attempts), and for mini mostly assembly and attribute errors (62 vs. 35).

Ablation: wrapper vs. predicates. At a reviewer's request we added C′, the same JSON wrapper, IDs and grouping as C but with each rule holding the prose sentence (Table V; post-hoc, protocol addendum frozen before its 625 trials). The wrapper alone (C′−B) cost −5.6, −4.1 and −6.6 pp (significant for nano, Holm $p = 0.008$); formal predicates (C−C′) cost a further −0.4, −3.7 and −5.5 pp (n.s.). Both

TABLE IV
CONFOUND CHECK ON THE IDENTICAL 114 SYNTHETIC TASKS: STRUCTURED − PROSE, FIRST ATTEMPT (PP, 95% CLUSTER CI). STUDY 1: SCHEMA-ONLY SYNTAX IN THE CONTRACT, 1.31× LONGER. STUDY 2: SAME INSTANCES, 0.98× LENGTH. 4.1 IN STUDY 2: 39 TASKS COMPLETED WITHIN BUDGET.

Model	Study 1 (unmatched)	Study 2 (matched)
nano	−39.5 [−52.6, −27.2]	−13.2 [−27.2, 0.0]
mini	−13.2 [−25.4, −1.7]	−11.4 [−21.1, −3.5]
4.1	+2.6 [−7.0, +12.3]	−2.6 [−8.1, 0.0]

TABLE V
ABLATION (POST-HOC): FIRST-ATTEMPT CORRECTNESS (%) FOR PROSE (B), JSON WITH PROSE SENTENCES (C′) AND JSON WITH FORMAL PREDICATES (C); PAIRED CONTRASTS IN PP, 95% CLUSTER CI, (HOLM p OVER THE TWO CONTRASTS). 4.1: ITS 91 TASKS.

Model	n	B	C′	C	C′−B (wrapper)	C−C′ (predicates)
nano	267	22.8	17.2	16.9	-5.6 [-9.9, -1.9] (0.008)	-0.4 [-5.9, +5.0] (1.000)
mini	267	41.2	37.1	33.3	-4.1 [-7.8, -0.4] (0.083)	-3.7 [-7.9, +0.0] (0.083)
4.1	91	56.0	49.5	44.0	-6.6 [-13.5, +0.0] (0.219)	-5.5 [-11.2, +0.0] (0.219)

contribute, so our conclusion concerns the tested presentation, not machine-readable contracts in general; C′ is also 24% longer than prose (1,807 vs. 1,455 tokens), which the wrapper effect cannot be separated from.

Confound check. Matching instances and length changed the conclusions of Study 1 (Table IV, identical synthetic tasks). Nano's penalty shrank from −39.5 to −13.2 pp, so most of the Study 1 collapse came from the unmatched instances and length, which changed together; its Study 1 failures (bracket lists instead of the documented `array<i64: ...>` schema) point to instantiation. Mini's prose advantage was unchanged (−13.2 vs. −11.4 pp), so a format effect remains. For 4.1, the effect moved from +2.6 to −2.6 pp (both n.s.), so Study 1's "structure helps the strongest model" did not survive. The comparison is between studies, not randomized within one.

F3: On-demand access trailed upfront delivery. D−C was −4.1 (n.s.), −13.5 and −31.9 pp. We identify three causes: uptake, suppressed lookup, and which facts are fetched. *Uptake:* 4.1 never called the retrieval tool (0 of 91 tasks), replicating Study 1, while nano and mini averaged 1.3 and 2.4 calls. Forcing tested this explanation and helped only the model that would not ask: requiring a first-turn call (D*) lifted 4.1 by +17.6 pp ($p = 0.007$; Study 1 post-hoc: +21.1), but left mini and nano unchanged and still trailed C by 14 pp. *Suppressed lookup:* giving the cross-cutting rules upfront (D^+) helped 4.1 (+17.6 pp over D, $p < 0.001$), but nano then almost stopped retrieving (0.02 calls per trial, down from 1.34) and lost −9.0 pp ($p < 0.001$). For every model, D^+ remained 13–14 pp below C.

Which facts are fetched. When a retrieving agent fetched the cross-cutting `module` facts, which include the attribute-encoding rule, it was correct 62% (nano), 51% (mini) and 57% (4.1, forced) of the time; when it did not, 7%, 2% and 16%. Without that rule, 44% of forced 4.1 attempts failed on attribute encoding, and only 24–52% of retrieving trials requested it. This is observational, because agents chose what to fetch, but it replicates Study 1 (88% vs. 22%). Together with D^+, it suggests that agents need both the cross-cutting rules and the operation facts, and that, left to choose, they often fail to fetch the cross-cutting rule. On-demand access was not cheaper either: forced retrieval used 7% *more* input tokens per trial than C for 4.1 (7.4k vs. 6.9k), with 14 pp lower accuracy.

F4: The observed bottleneck was well-formedness. Across models, 99.2–99.8% of failed first attempts were rejected by the parser, the verifier or the signature check. This held on the KernelBench suite too: at most 0.5% of its failed first attempts verified and then produced wrong values, so harder, real-world tasks lowered correctness without making wrong values the dominant failure. Programs that fail verification never reach semantic testing, so this describes where agents failed, not whether semantic reasoning matters. Only 15 of the 5,000 analyzed first attempts executed and produced wrong values. The failure mix was dominated by invalid assembly syntax (33–44%), shape/type verifier errors (14–27%), malformed literals (16–22%) and attribute encoding (13–16%). Fig. 1 (right) shows a typical case: under D^+, mini had the encoding rule upfront, still wrote the legacy `dense` form, and repaired it after the verifier diagnostic. Up to two repairs added 0.3–14.2 pp, with the largest gain for mini under D^+.

V. IMPLICATIONS FOR AGENT-FACING VIRTUAL ISAS

The following are hypotheses supported for one model family and one IR, stated as design guidance to be re-tested on others. **Validate interface claims with matched treatments.** The same comparison gave opposite answers before and after we equalized content, examples and length, so agent-facing ISA documentation should be evaluated with fact-equivalent, length-matched variants. **Test contract formats before adopting them.** The tested JSON contract scored at or below prose for every model, and both its wrapper and its formal predicates contributed. An ISA can generate several renderings from one fact inventory and should measure which one its agents use best. **Do not rely on agents to ask.** Strong agents skipped retrieval, partially documented agents stopped retrieving, and retrieving agents often missed the cross-cutting rule. For self-describing virtual ISAs, introspection mechanisms therefore need uptake design, such as a required first query or diagnostics that cite the violated fact. Returning cross-cutting rules with every operation-level answer is a further option that F3 motivates but we did not test. **Make well-formedness learnable.** Most failures are static, so a normative, version-pinned surface syntax with concrete instances, plus diagnostics that name the violated rule, targets the actual bottleneck.

VI. THREATS TO VALIDITY

Models. We used three snapshots of one family (GPT-4.1, April 2025) at temperature 0, with one trial per cell; API outputs at temperature 0 are not fully deterministic, so each cell carries sampling noise. Behaviors such as never calling

a tool may be habits of this family, so every implication is a hypothesis for other families. Model 4.1 covers 91 of 267 tasks because of the budget cap; that subset has the same suite mix and is slightly harder for nano and mini, whose C−B estimates on it keep their sign (-3.3, -6.6 pp). *Tasks.* Synthetic tasks are small and fully specified, and KernelBench tasks were shrunk and exclude large 3D transposed convolutions. The original KernelBench oracle cases are non-negative; the stronger re-grade (§III-C) changed 1 first-attempt and 3 after-repair labels and none of the 24 pre-registered contrasts by more than 0.5 pp. KernelBench's PyTorch sources are public and may be in the models' training data; only the StableHLO lowering is new. Because almost all failures are static, these results speak to syntactic and structural programmability more than to semantic reasoning. *Treatments.* The fact inventory and audit were authored with AI assistance (see the disclosure below); the automated audit is necessary but not sufficient, and an independent human audit of a stratified 24-fact sample (audit sheet released) is pending. Prose and contract differ in notation by design (sentences vs. predicates), and our claim concerns that difference at matched content and length. *Agent loop.* Repairs are harness-driven. A self-directed agent track (verify, run-example and submit tools) is implemented but was not evaluated, so we make no claims about autonomous agents. *Procedure.* Two robustness fixes during Study 2 (API key precedence, and a call watchdog after network stalls) changed no prompt, grading or analysis; both are recorded as dated addenda.

VII. Conclusion

We have shown that, holding the IR, toolchain, tasks and agent fixed, the agent-facing interface of a virtual ISA changed correctness by tens of percentage points. Documentation presence dominated; at matched content, prose scored at or above the tested JSON contract; and on-demand access failed unless uptake was designed in. We have also shown that unmatched documentation comparisons can reverse these conclusions. Agent-facing virtual ISAs should therefore be engineered and measured as interfaces, with matched controlled experiments, and our next step is to test these hypotheses on other model families and on a closed-loop agent.

Artifact Availability

The harness, fact inventory, task generator and KernelBench converter, oracles, frozen protocols with hashes, and all trial logs and transcripts are in a repository that is currently private. They are available to reviewers on request and will be made public upon publication.

AI-Use Disclosure

The author used an AI coding assistant (Anthropic Claude, via Claude Code) throughout this work. It reviewed the initial research scaffold; implemented the harness, evaluator, task generator, KernelBench converter and analysis scripts; drafted the fact inventory from the pinned specification; ran the experiments (Study 2 and the ablation within budgets the author approved; the Study 1 spend of $6.80 was incurred before a budget was set); and drafted and revised the manuscript text, tables and figures. All citations were checked against publisher, proceedings or arXiv records. The GPT-4.1 models are the objects of study, not authoring tools. The author directed the study, made the design and budget decisions, and takes full responsibility for the content.

References

[1] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization (CGO)*, 2004, pp. 75–86.

[2] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "MLIR: Scaling compiler infrastructure for domain specific computation," in *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021, pp. 2–14.

[3] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini, "KernelBench: Can LLMs write efficient GPU kernels?" in *Proceedings of the 42nd International Conference on Machine Learning (ICML), PMLR 267*, 2025, pp. 47 356–47 415.

[4] J. Li, S. Li, Z. Gao, Q. Shi, Y. Li, Z. Wang, J. Huang, H. Wang, J. Wang, X. Han, Z. Liu, and M. Sun, "TritonBench: Benchmarking large language model capabilities for generating Triton operators," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 23 053–23 066.

[5] Y. Guan, Y. Lin, X. Zhao, J. Yao, X. Qiang, Z. Yu, P. Viswanath, Y. Ding, and A. Aziz, "TritonGym: A benchmark for agentic LLM workflows in Triton GPU code generation," in *Proceedings of the 43rd International Conference on Machine Learning (ICML), PMLR 306*, 2026, pp. 37 457–37 473.

[6] Z. Yang, H. Ding, F. Liu, Y. Cheng, H. Cheng, Z. Chai, and H. Zhou, "CIRBench: Evaluating large language models as LLVM IR optimizers," in *Proceedings of the 43rd International Conference on Machine Learning (ICML), PMLR 306*, 2026, pp. 144 491–144 544.

[7] J. Nie, H. Wu, Y. Lai, Z. Cao, C. Zhang, B. Lou, E. Wang, J. Cheng, T. M. Jones, R. D. Mullins, R. Antonova, and Y. Zhao, "KernelCraft: Benchmarking for agentic close-to-metal kernel generation on emerging hardware," in *Proceedings of the 43rd International Conference on Machine Learning (ICML), PMLR 306*, 2026, pp. 93 070–93 121.

[8] OpenXLA, "StableHLO specification," https://github.com/openxla/stablehlo/blob/23d2f926/docs/spec.md, commit 23d2f926, accessed 2026-10-13.

[9] P. K. Rath, "Cross-dialect generalization without retraining: Benchmarks and evaluation of schema-derived constrained decoding for MLIR," 2026, arXiv:2607.18254.

[10] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 50 528–50 652.

[11] S. Zhou, U. Alon, F. F. Xu, Z. Wang, Z. Jiang, and G. Neubig, "DocPrompting: Generating code by retrieving the docs," in *International Conference on Learning Representations (ICLR)*, 2023.

[12] R. Wang and L. Zhang, "Documentation retrieval improves planning language generation," in *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (IJCNLP-AACL)*, 2025, pp. 147–158.

[13] A. N. Ashik, S. Wang, T.-H. Chen, M. Asaduzzaman, and Y. Tian, "When LLMs lag behind: Knowledge conflicts from evolving APIs in code generation," 2026, arXiv:2604.09515.

[14] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, "Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting," in *International Conference on Learning Representations (ICLR)*, 2024.

[15] J. He, M. Rungta, D. Koleczek, A. Sekhon, F. X. Wang, and S. Hasan, "Does prompt formatting have any impact on LLM performance?" 2024, arXiv:2411.10541.